

Introduction:

The report describes the implementation of a memory management system for a simulated virtual machine(VM), with support for process management. The system involves handling processes, allocating and deallocating memory, and managing page tables to simulate a real OS environment. There are no additional member functions that is used during the implementation.

System Overview:

The system supports the following functionalities:

- **Process Management:** Creation, loading and execution of processes.
- **Memory Management:** Allocation and deallocation of virtual memory pages
- **Page Table Management:** Mapping virtual pages to physical memory frames
- **Context Switching:** Switching between different running processes by using `tyld()`

Implementation Details:

Memory Management:

AllocMem(): The aim of this function is to allocate free physical page for a given virtual page number(VPN). It checks if the page is already allocated and if there are free pages available using a bitmap. If the function is able to find a free page in the free bitmap then it allocates the memory else it returns a message saying there is not enough space. The function also returns an integer value to inform the user if the allocation was successful or not.

FreeMem(): This function frees a previously allocated physical page and updates the free bit map. To do this we first extract VPN and offset, and then find the PFN by using the page table entry. And then the function marks the Page as free in the FreeBitmap.

Process Creation and Loading:

CreateProc(): This function creates a new process by allocating memory for code and heap segments, initializing the Process Control Block(PCB), and assigning a unique process id (PID) which starts from 0. During the implementation of this function I faced a lot of problems, it was really hard for me to debug the code to see what was wrong and the whole code depends on the correctness of this function which showcases it's importance.

LoadProc(): The aim of this function is to load a process into CPU registers using the information stored in its PCB. This was the easiest function to implement in the code.

Memory Access and Segmentation Faults:

mr(): This function does memory reads, if we try to access illegal addresses then we display a segmentation fault error. All of these are done by using the input address passed as an argument. The function first extracts the vpn, and the offset and calculates the physical memory using them(also by using the page table entry to find the physical address corresponding to this vpn). Then it checks whether we are trying to access an unallocated memory if so we return a segmentation fault error else we read the value.

mw(): This function handles memory writes, if we have a write permission on the given address then the function successfully writes else the function displays an error. The same idea in mr() is used here, the only difference we do additional checks on whether we have a read or write permissions.

Trap Handling:

Tbrk(): This function handles memory allocation and deallocation requests from a running process. We extract the informations from the R0 register. This was also hard for me to debug, I was making a mistake to extract the VPN(I did not think I made calculation mistakes to extract the vpn from the register) hence I lost a lot of time during debug.

Thalt(): This function terminates the current process and initiates a context switch to the next available process. The method looks for processes to halt, and we initiate a context switch after halting the current process. If there are no other processes running we set running to false to indicate that the program is over.

Tyld(): Handles context switching, which is similar to yield() function to yield a process. If there is a single process this process keeps running after calling tyld(). This method finds the next runnable process if it exists, if we are able to find it we switch to that process and load its contents to the registers and memory.

Challenges and Solutions:

Problem 1:

```
static inline void tbrk() {
    uint16_t r0 = reg[R0]; // getting the value of R0 register
    //uint16_t vpn = r0 & 0x1F; // Extract VPN (first 5 bits)
    //0xF800
    uint16_t vpn = (r0 >> 11) & 0x1F; // Extract VPN from the most significant 5 bits
    // 0x1F = 0001 1111, hence when we do the AND operation we effectively extract the first 5 bits
    bool write = (r0 >> 2) & 1; // Extract write bit (specified in the homework doc)
    bool read = (r0 >> 1) & 1; // Extract read bit (specified in the homework doc)
    bool allocate = r0 & 1; // Extract allocation/freeing bit (specified in the homework doc)
    uint16_t curProcID = mem[Cur_Proc_ID]; // Get the current process ID
```

In here I was calculating VPN incorrectly, I never thought I would make such a mistake but after hours of debugging I figured out that I made this mistake here, and fixed it. The issue was I was extracting the VPN starting from the LSB(least significant bit) but I should have started from MSB(Most significant bit).

Problem 2: I was calculating ptbr incorrectly and that led to making mistakes in my createProc function. More specifically I was updating the ptbr in terms of bytes but I should I have done in terms of words.

The 2 problems are just a couple of them, these are the ones that consumed a lot of time to fix. The rest of the problems were minor details but easy to see.

Conclusion:

The implementation provides a basic but functional simulation of an operating system's memory management and process handling. The system is capable of managing processes, allocating memory, handling traps, and performing context switches, simulating a simple virtual machine environment.

